

ZSA Guide

Issuance, private transfer, and public supply

m@rek.onl

Abstract

A shielded asset is not a number with a hidden text label. Its identity must survive issuance, commitments, transfers, splitting, and burning, without letting an adversary invent algebraic exchange rates between unrelated assets. This volume constructs that chain of obligations. It derives the issuer's signature, the public issuance state machine, the private per-asset balance equation, and the membership rule that makes zero-value split inputs safe. The endpoint is the mathematical and validation design in one pinned pair of draft ZIPs. Where those drafts leave their transaction carrier or note-format instantiation unfinished, the boundary remains visible.

Contents

1	The asset ledger contract	3
1.1	What changes when there is more than one asset	3
1.2	The selected endpoint	3
1.3	Dependencies and notation	3
1.4	State is branch-local	4
2	Asset identity and issuance authority	4
2.1	Deriving a separate issuance key	4
2.2	The secp256k1 instance	5
2.3	Tagged hashing and a complete issuance signature	5
2.4	From a description to an asset base	6
2.5	The algebraic requirement on asset generation	6
3	Issuance state and issue notes	7
3.1	The public creation record	7
3.2	Freshness derived from the transaction	7
3.3	The reference note	8
3.4	The state transition	8
3.5	Supply and finalisation invariants	9
3.6	Appending the commitments	9
4	Asset-bearing notes and conservation	9
4.1	Binding the asset into the note	9
4.2	Randomness and the remaining note-format parameter	10
4.3	A private multibase value commitment	10
4.4	The transaction equation	10
4.5	From field balance back to integer balance	11
4.6	A two-asset check	12
5	Split inputs and the counterfeiting boundary	12
5.1	Why ordinary zero-value padding is insufficient	12
5.2	Authenticate the base without consuming value	12
5.3	A fresh nullifier for the non-spending copy	13
5.4	Provenance is an inductive invariant	13
6	The Action relation and transaction validation	14
6.1	Public statement and private witnesses	14
6.2	How the Boolean conditions become field equations	14
6.3	Complete validation order at the defined interface	15
6.4	What is still needed for a deployed endpoint	16

1 The asset ledger contract

The integer, field, and point-encoding language used here comes from *Math Guide*, §*Prime fields and modular arithmetic* and §*Pasta curves and their encodings*. Probability, including uniform randomness, is constructed in *Math Guide*, §*Probability and computation*. *Crypto Guide*, §*Security contracts and reductions*, defines computational security and failure probabilities. Its §*Groups, commitments, and Sinsemilla* defines hiding and binding and constructs the commitments used by the inherited note relation; §*Knowledge, simulation, and Schnorr* defines knowledge and extraction.

1.1 What changes when there is more than one asset

In the native payment protocol, every amount uses the same unit: a zatoshi. If one input holds 80 000, outputs hold 50 000 and 20 000, and the fee is 10 000, the equation

$$80\,000 = 50\,000 + 20\,000 + 10\,000$$

has a meaning before any cryptography is applied.

For two assets A and B , the expression $80A = 50B + 30A$ is not a balanced payment. No exchange rate is part of the shielded ledger's addition operation. A valid transaction must conserve each asset separately, except for explicitly authorised creation or public destruction of that same asset.

The design separates three operations.

1. *Issuance* creates custom-asset notes. The asset, recipient, amount, and note randomness are publicly available, and an issuer signature authorises the creation.
2. *Transfer* consumes notes and creates notes. The proof hides their assets and amounts while enforcing per-asset balance.
3. *Burn* removes a public amount of a named custom asset from the shielded ledger. The proof and binding signature account for that removal.

An issuer controls further creation of its assets, not the spend authorisation keys of holders. The native asset retains its existing value base and pays transaction fees. A custom asset cannot be moved into a transparent native-value field by relabelling its units.

1.2 The selected endpoint

The rule set here is ZIP 226 and ZIP 227 at commit 69610984. They are draft specifications, not evidence that these rules are active on a network. Their Orchard definitions incorporate the ZIP 2005 changes at that same source pin.

The asset mechanisms are substantially specified, but the pair still references the withdrawn ZIP 230 transaction carrier and ZIP 246 digest scheme. Its ZSA note lead byte is a placeholder. Consequently, this volume does not claim to define an interoperable, deployable ZSA transaction encoding. It constructs the specified state and proof contracts and identifies exactly which missing interfaces prevent turning them into a complete byte-level validator. The version-six transaction in the core volumes is not silently extended with these fields.

1.3 Dependencies and notation

Ironwood Guide, §*Actions and the proof relation*, supplies the one-input/one-output Action, the note commitment, Merkle membership, nullifier, and ownership relation. Its §*Balance, digests, and authorisation* supplies the binding signature and the distinction between transaction effects and authorising data. We change those constructions only where stated.

Math Guide, §*Pasta curves and their encodings*, supplies Pallas. Write $\mathbb{P}$ for its prime-order group, p for its base-field modulus, and q for its group order. Thus note values such as ρ, ψ, nf lie in $\mathbb{F}_p$, whereas commitment randomness and signature scalars lie in $\mathbb{F}_q$. These two fields must not be interchanged.

Write V for the existing Orchard native-value base and R_v for its value-commitment randomness base. An asset base A is a nonidentity Pallas point. The point's canonical encoding A^* is 32 bytes; when fed to a bit-oriented commitment, these same bytes are unpacked least-significant bit first. $\text{enc}_k(x)$ means exactly k bytes of little-endian integer encoding. The shared notation $\text{LE}_k(x)$ instead denotes exactly k bits, least-significant bit first.

1.4 State is branch-local

A full validator keeps the existing note tree, recognised anchors, and spent-nullifier set. It additionally keeps an issuance map

$$I[A] = (\text{balance}, \text{final}, \text{reference}).$$

The map is part of the state of a particular chain branch. Processing a transaction produces a candidate next state; any failed check discards the entire candidate. A reorganisation restores the state at the common ancestor and replays the replacement branch.

In particular, an issuance finalisation is irreversible *along a fixed accepted branch*, not immune to a reorganisation of that branch. No extra finality assumption is hidden in the word final.

2 Asset identity and issuance authority

2.1 Deriving a separate issuance key

The issuance secret is independent of the protocol role of a note spending key. A holder may authorise a payment without gaining authority to issue more units, and an issuer may sign issuance transactions without learning holders' spend keys.

The derivation reuses the hardened-only machinery in *Wallet Guide*, §*From seed to account*, with a different master domain and path. Let the seed S contain between 32 and 252 bytes; it should contain at least 256 bits of entropy. For the guessing threat here, a conservative meaning is that no one seed occurs with probability greater than 2^{-256} , called at least 256 bits of *min-entropy*. Merely storing 32 bytes does not ensure this unpredictability. Set

$$(s_0, c_0) = \text{split}_{32}(\text{BLAKE2b}_{512}(\text{ZcashSA_Issue_V1}, S)).$$

The first half is a 32-byte secret and the second is a chain code. For a hardened index $i \geq 2^{31}$, define

$$(s', c') = \text{split}_{32}(\text{PRFexpand}_c(0x81 \parallel s \parallel \text{enc}_4(i))).$$

The byte 0x81 is hexadecimal; the empty optional lead and tag arguments of ZIP 32 contribute no bytes here. Apply this step along

$$m_{\text{issuance}}/227'/\text{coin_type}'/0'.$$

On mainnet the coin-type index is 133'. The account index is fixed to zero in this selected design. The final 32-byte secret is isk .

This is derivation of a *byte string*. Interpreting that string as the secp256k1 secret scalar below uses big-endian order. It is not the Pallas little-endian scalar conversion used by RedPallas. If the result is not a valid secp256k1 secret, key derivation fails at the public-key step. ZIP 227 calls for discarding the key and using a different issuance secret; it does not specify an automatic hierarchical-key-derivation retry convention preserving the fixed path.

2.2 The secp256k1 instance

Issuance uses the Schnorr construction from *Crypto Guide*, §*Knowledge, simulation, and Schnorr*, instantiated by BIP 340. This is a different curve from Pallas.

The curve is

$$y^2 = x^3 + 7 \pmod{p_s}, \quad p_s = 2^{256} - 2^{32} - 977,$$

and its group order is the integer whose hexadecimal encoding is

$$n_s = \text{FF} \\ \text{BAAEDCE6AF48A03BBFD25E8CD0364141}.$$

The displayed lines concatenate; they are not an addition. The standard generator G_s has coordinates

$$x(G_s) = 79BE667EF9DCBBAC55A06295CE870B0702 \\ 9BFCDB2DCE28D959F2815B16F81798, \\ y(G_s) = 483ADA7726A3C4655DA4FBFC0E1108A8FD \\ 17B448A68554199C47D08FFB10D4B8.$$

Again concatenate each coordinate's lines. The arithmetic formulas are those of *Math Guide*, §*Pasta curves and their encodings*, with the curve constant changed from 5 to 7. The curve's prime group order and discrete-logarithm hardness are properties and assumptions of this selected standard curve, not consequences of merely writing its equation.

Let BE_{32} encode an integer in exactly 32 big-endian bytes. A public key contains only the x -coordinate of its point. To decode it, reject $x \geq p_s$, compute

$$c = x^3 + 7 \pmod{p_s}, \quad y = c^{(p_s+1)/4} \pmod{p_s},$$

and reject unless $y^2 = c$. Of y and $-y \pmod{p_s}$, select the even representative. The exponent is a square-root candidate because $p_s \equiv 3 \pmod{4}$; the explicit square check rejects non-residues. This operation is lift_x .

Given $d' = \text{int}_{\text{BE}}(\text{isk})$, reject unless $1 \leq d' < n_s$. The public key is $\text{ik} = \text{BE}_{32}(x([d']G_s))$. For signing, replace d' by $n_s - d'$ if its public point has odd y . The resulting d satisfies $[d]G_s = \text{lift}_x(\text{ik})$.

2.3 Tagged hashing and a complete issuance signature

Crypto Guide, §*Hash functions and key derivation*, constructs SHA-256. BIP 340 gives each use a tagged wrapper:

$$H_{\text{tag}}(m) = \text{SHA256}(\text{SHA256}(\text{tag}) \parallel \text{SHA256}(\text{tag}) \parallel m),$$

where the tag is its literal UTF-8 byte sequence. For the issuance transaction's 32-byte signature digest M , the selected signing algorithm is as follows.

1. Obtain the normalised scalar d and public key ik . Set

$$t = \text{BE}_{32}(d) \text{ xor } H_{\text{BIP0340/aux}}(0^{32}).$$

Here 0^{32} is 32 zero bytes, as required by ZIP 227.

2. Set

$$k' = \text{int}_{\text{BE}}(H_{\text{BIP0340/nonce}}(t \parallel \text{ik} \parallel M)) \pmod{n_s}.$$

Reject $k' = 0$. If $[k']G_s$ has odd y , replace k' by $n_s - k'$. Call the result k , and let $R = [k]G_s$.

3. Compute

$$e = \text{int}_{\text{BE}}(H_{\text{BIP0340/challenge}}(\text{BE}_{32}(x(R)) \parallel \text{ik} \parallel M)) \pmod{n_s}, \quad s = k + ed \pmod{n_s}.$$

Form $\sigma = \text{BE}_{32}(x(R)) \parallel \text{BE}_{32}(s)$. Verify it with the following algorithm before returning it; abort if this self-check fails.

Verification first checks the public-key and signature lengths. Lift the public key to the even- y point P . Parse the signature as integers r, s , and reject $r \geq p_s$ or $s \geq n_s$. Recompute e from $\text{BE}_{32}(r)$, the public key, and M . Set $R' = [s]G_s - [e]P$. Accept precisely when R' is nonidentity, has even y , and has $x(R') = r$.

Correctness is the identity

$$[k + ed]G_s - [e]([d]G_s) = [k]G_s.$$

The parity normalisations ensure that the verifier uses the same representatives as the signer. Security additionally requires the computational and hash assumptions of Schnorr; correctness alone does not imply unforgeability. Fixed auxiliary bytes do not make the nonce public: its derivation includes the secret d . They do remove the additional fault and side-channel protection that fresh auxiliary randomness can provide.

The scheme signature σ is 64 bytes. At ZIP 227's scheme-tagged interface,

$$\text{issuer} = 0x00 \parallel \text{ik}, \quad \text{issueAuthSig} = 0x00 \parallel \sigma,$$

which are respectively 33 and 65 bytes. This does not specify the outer transaction field framing; that is part of the carrier boundary discussed later.

2.4 From a description to an asset base

An asset description D is a nonempty byte sequence, preferably well-formed UTF-8. Consensus stores its digest, not a requirement to publish a particular human-readable description. Define

$$\begin{aligned} h_D &= \text{BLAKE2b}_{256}(\text{ZSA-AssetDescCRH}, D), \\ \text{AssetId} &= (\text{issuer}, h_D), \\ B_{\text{id}} &= 0x00 \parallel \text{issuer} \parallel h_D, \\ h_A &= \text{BLAKE2b}_{512}(\text{ZSA-Asset-Digest}, B_{\text{id}}), \\ A &= \text{GroupHash}^{\mathbb{P}}(\text{z.cash:OrchardZSA}, h_A). \end{aligned}$$

The BLAKE2b personalisation strings are zero-padded to their 16-byte parameter fields, as in *Crypto Guide*, §Hash functions and key derivation, which also constructs the exact Pallas GroupHash.

The first byte of B_{id} is an asset-identifier version. The next byte is the signature-scheme byte inside the issuer. They happen both to be zero, but perform different jobs. The total length is $1 + 33 + 32 = 66$ bytes. h_A has 64 bytes, and A^* has 32 bytes.

An asset name is not an identity. Two issuers may hash the same text and still obtain different identifiers because their issuer fields differ. Conversely, two bitwise different descriptions chosen to collide in h_D would define the same identifier under one issuer. Avoiding that attack is a collision-resistance assumption, not a type-system guarantee. Wallets must identify assets by authenticated identifiers or user-established names bound to them, not by an untrusted description alone.

2.5 The algebraic requirement on asset generation

All nonidentity Pallas points generate the same cyclic group. There is therefore some scalar a such that $A = [a]V$, and some scalar relation between any two asset bases. The construction does not make those relations cease to exist. It must make useful nontrivial relations computationally infeasible to find.

In particular, issuers must not choose a scalar a themselves and declare $[a]V$ to be their asset base. Their choice is an identifier preimage; the independently domain-separated group hash determines the base. The later conservation argument assumes that an efficient adversary cannot find a nonzero relation among these derived bases and R_v , within its permitted choices and queries. Modelling the

hash-to-group outputs as independent random group points gives the usual discrete-logarithm-based justification. The concrete hash is an instantiation of that model, not a proof of its own hardness.

The map from 512-bit digests to a roughly 255-bit group cannot be injective. Nor is the group hash's output type, by itself, a proof that it avoids the identity or the native base. The note type requires a nonidentity point, and native V has its own branch. A complete deployment must resolve exceptional generated points and asset-identity collisions consistently. It must not treat a negligible-probability edge case as an unspecified parser choice.

3 Issuance state and issue notes

3.1 The public creation record

An issue note contains

$$(d, \text{pk}_d, v, A, \rho, \text{rseed}).$$

Here d is an 11-byte diversifier, pk_d a nonidentity diversified transmission key, $v \in [0, 2^{64} - 1]$ an integer amount, A the asset base, $\rho \in \mathbb{F}_p$, and rseed 32 bytes of note randomness. The recipient construction is inherited from *Ironwood Guide*, §*Keys and viewing capabilities*, and *Wallet Guide*, §*Addresses and receiver selection*.

Unlike an encrypted transfer output, these issue-note fields are available to validators and chain observers. Anyone can reconstruct its commitment once the note-format derivations are instantiated. The issuer cannot conceal a larger minted value inside the commitment while publicly accounting for a smaller one: the validator recomputes the commitment from the public amount.

The recipient's spending authority is still private for an ordinary issue note. Knowing its commitment opening is not knowing its nullifier key or spending key. Later unlinkability nevertheless depends on the inherited nullifier, proof, and transaction-privacy assumptions; it is not an unconditional consequence of putting the note into a common tree.

3.2 Freshness derived from the transaction

An *Action Group* is an ordered nonempty list of transfer Actions sharing a public anchor, spend/output/custom-asset enable flags, an expiry bound, a public burn list, and a proof covering the group; each Action also has its spend-authorisation signature. This is the logical grouping required by the selected design's referenced carrier, not a claim that the withdrawn carrier supplies a deployable encoding. The selected issuance rules allow at most one such group.

Every issuance bundle accompanies a transfer bundle with an Action from which a nullifier can be taken. Let n_0 be the nullifier of the first Action in the first Action Group. For issuance Action index i and note index j , both zero-based 32-bit integers, the issued note uses

$$\rho_{i,j} = \text{ToBase}(\text{PRFexpand}_{\text{enc}_{32}(n_0)}(0x84 \parallel \text{enc}_4(i) \parallel \text{enc}_4(j))).$$

Here ToBase interprets the 64 output bytes little-endian and reduces modulo p . The first byte is $0x84$.

This gives each note a transaction-dependent origin, rather than letting an issuer reuse an arbitrary ρ in several transactions. The input-nullifier uniqueness rules and the transaction signature bind the construction to the containing transaction. The PRF's key is public in this use; secrecy is not its purpose here. Collision avoidance for derived ρ values is a computational hash requirement, not literal injectivity of reduction modulo p .

An *IssueAction* contains a description hash h_D , a finalisation flag, and an ordered list of issue notes. An issuance bundle contains one issuer identifier, an ordered list of those Actions, and the issuer's signature over the *whole transaction* signature digest. It is not merely a signature over the issuance amounts.

3.3 The reference note

For the first issuance of an asset, the first note must have value zero and the default external diversified address derived from the all-zero 32-byte Orchard spending key, at diversifier index zero. Its address bytes are the following concatenation:

```
cc36601959213b6b0cdb96a75c17c3a668a97f0d6a
8c5ce164a518ea9ba9a50ea75191fd861b0ff10e62b0.
```

These 43 bytes are $d \parallel \text{pk}_d^*$. The derivation, rather than a visual resemblance to a human-readable address, gives the meaning of this constant.

The reference is a real committed leaf but holds no value. Its note data and its deliberately public spending key are known. After its inclusion and an accepted anchor are available, anyone can use the corresponding witness material to demonstrate the existence of the asset in the split construction below. This explains how that construction can obtain a publicly usable membership witness without publishing a value-bearing spend key.

The reference's publicly known keys make it an exception to the ordinary issue note's private-key argument. It must never be used as a recipient for funds that are meant to be owned exclusively by one person. Copying its opening does not mint units: a split input contributes zero to the value equation.

3.4 The state transition

For a missing asset entry, read

$$I[A] = (0, \text{false}, \perp), \quad \text{MAX_ISSUE} = 2^{64} - 1.$$

The balance records *issued minus burnt* units. It is a cap on circulating ledger supply, not on the sum of all issuance events over the asset's lifetime.

Start transaction validation with a working copy $J \leftarrow I$. The burn set contains distinct pairs (A, b_A) , with

$$A \neq V, \quad A \neq \mathcal{O}, \quad 1 \leq b_A \leq 2^{63} - 1.$$

Every point must decode canonically. There may not be two burn entries for the same base.

Process all burns before any issuance:

1. For each (A, b_A) , require $J[A].\text{balance} \geq b_A$.
2. Replace that balance by $J[A].\text{balance} - b_A$.

The transaction's proof and binding signature, not this public subtraction alone, establish that the sender actually supplies the burnt units.

If there is an issuance bundle, require the accompanying Action Group and a valid issuer signature over the transaction digest. Process the issuance Actions in order:

1. Derive A from the bundle's issuer and the Action's h_D . Require $J[A].\text{final} = \text{false}$.
2. If $J[A].\text{reference} = \perp$, require a first note and check that it is the zero-valued reference note just specified. Store it as $J[A].\text{reference}$.
3. For each note in order, validate its recipient and field encodings, compute its required $\rho_{i,j}$, and check its equality with the supplied ρ . Construct its note commitment from its public fields and the selected note derivations, rejecting any undefined commitment operation.
4. Before adding v , require $J[A].\text{balance} + v \leq \text{MAX_ISSUE}$ as an integer inequality. Add v with checked arithmetic.
5. If the Action finalises issuance, set $J[A].\text{final} \leftarrow \text{true}$.

Two issuance Actions for one asset are not made independent by sharing a bundle. The second observes the first's state changes. If the first finalises, a later Action for that asset fails. An empty list cannot supply the mandatory first reference for a new asset.

Only if the entire transaction passes all its proof, signature, encoding, nullifier, anchor, and state checks does J become the new issuance state. At activation the map is empty. Within a block, each transaction sees the previous transaction's output map; the first sees the parent's final map.

3.5 Supply and finalisation invariants

For one asset and any accepted branch prefix, let M_A be the sum of public issue-note values and B_A the sum of public burn values in that prefix. By induction on the ordered operations,

$$I[A].\text{balance} = M_A - B_A, \quad 0 \leq M_A - B_A \leq 2^{64} - 1.$$

The base case is the empty map. A burn subtracts the same amount from both expressions, with its lower-bound check preserving nonnegativity. An issuance adds the same amount, with its checked upper bound preserving the cap. Transfer-only transactions alter neither side. This proves the public accounting invariant exactly.

It does *not* yet prove that privately spendable notes have that total value. That stronger statement also requires the transfer conservation argument, proof knowledge soundness, note binding, and the nullifier uniqueness rules.

For example, issue 1000 units, burn 100, then issue 50. The state balances are 1000, 900, 950, while lifetime issuance is 1050. At a full supply cap, burning b and then issuing b in one transaction is compatible with the cap, provided the asset has not been finalised. Reversing the specified operation order could change acceptance at the cap.

Finalisation is a monotone Boolean along a branch. Once true, no later issuance Action for that asset is permitted. Burning remains possible and does not clear the flag. Thus finalisation limits future creation, not holders' transfers, and the outstanding supply can still decrease.

3.6 Appending the commitments

The new tree leaves are ordered independently of the order in which public supply changes are evaluated:

1. Append transfer-output commitments in Action Group order and Action order.
2. Append issue-note commitments in issuance Action order and note order.

The reference note is included in the second part like any other issue note. Commitment positions, paths, and anchors must agree with this order. A public issue-note description is not a substitute for a Merkle path against a recognised anchor.

4 Asset-bearing notes and conservation

4.1 Binding the asset into the note

The conceptual note relation now opens

$$(d, \text{pk}_d, v, A, \rho, \psi, \text{rcm}).$$

For the native base $A = V$, its commitment is the inherited Orchard commitment, without an appended asset field.

For a custom asset define the bit string

$$m_A = g_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi) \parallel A^*,$$

where point encodings in this expression are their 256-bit forms. Then

$$\begin{aligned} \text{cm}_A &= \text{SinsemillaHashToPoint}(\text{z.cash:ZSA-NoteCommit-M}, m_A) + [\text{rcm}]R_c, \\ R_c &= \text{GroupHash}^{\mathbb{P}}(\text{z.cash:Orchard-NoteCommit-r}, \epsilon). \end{aligned}$$

The empty string is ϵ . *Crypto Guide*, §Groups, commitments, and Sinsemilla, gives the bit chunking, incomplete-operation rejection, and the distinction between the full point and its extracted x -coordinate. The leaf is the inherited extracted commitment cm_x .

For a custom note, the input has $256 + 256 + 64 + 255 + 255 + 256 = 1342$ bits before Sinsemilla padding. Changing the asset changes committed input bits as well as the hash domain relative to a native note. The randomness base R_c is shared; the native branch remains byte-for-byte the native commitment construction.

The proof witnesses A once and uses it for the old note, the new note, and the value commitment. Three unconstrained copies merely bearing the same name would not enforce this requirement.

4.2 Randomness and the remaining note-format parameter

The note's ψ is derived by the inherited ZIP 2005 function

$$\psi = \text{ToBase}(\text{PRFexpand}_{\text{rseed}}(0x09 \parallel \text{enc}_{32}(\rho))).$$

The split-only randomness below deliberately uses $0x0A$, not this $0x09$ domain.

The selected ZIP pair calls the ZIP 2005 note-randomness derivation with its placeholder ZSA lead byte. The core rule distinguishes legacy $0x02$ randomness from recoverable $0x03$ randomness, but neither fact assigns a ZSA format byte. In particular, one cannot replace the placeholder by $0x03$ just because it once appeared in a withdrawn carrier.

Accordingly, the relation here takes a valid note opening (ρ, ψ, rcm) as witness data. Deriving rcm from the public rseed of an *issue note*, or constructing an interoperable encrypted transfer plaintext, additionally needs the missing ZSA note-format instantiation. This is a concrete boundary of the selected draft, not permission for validators to choose their preferred derivation.

4.3 A private multibase value commitment

For an ordinary Action with old and new values v_o, v_n , set

$$C = [v_o - v_n]A + [r]R_v.$$

The amount difference is a signed integer, subsequently interpreted modulo q for scalar multiplication. The circuit's range checks must establish that the note values are actual 64-bit integers before this scalar interpretation.

There is one randomness scalar r for each Action. If r is uniform in $\mathbb{F}_q$, then for any fixed A and value difference, C is uniform in $\mathbb{P}$: translation by the fixed amount term permutes the group. Thus this one commitment perfectly hides its amount term under an independently uniform mask.

That marginal statement is not a proof of joint transaction privacy. The transaction reveals a linear combination of commitments through its binding key, contains note commitments and ciphertexts, and may publish issuance or burn information. Security for all those correlated objects needs the inherited proof and encryption properties and appropriately joint randomness assumptions.

4.4 The transaction equation

Let $\Delta_i = v'_{o,i} - v_{n,i}$, where $v'_{o,i}$ is the effective input value: the real value for a normal input and zero for a split input. Then

$$C_i = [\Delta_i]A_i + [r_i]R_v.$$

Let b_{nat} be the native value leaving this bundle, including its contribution towards the transaction fee, and let (A, b_A) range over the public burn set. The binding public key is

$$K_{\text{bind}} = \sum_i C_i - [b_{\text{nat}}]V - \sum_A [b_A]A.$$

For an honest balanced transaction,

$$K_{\text{bind}} = [k_{\text{bind}}]R_v, \quad k_{\text{bind}} = \sum_i r_i.$$

The binding signature uses the inherited binding-signature generator R_v , not the spend-authorisation generator or an issuer key, and signs the complete transaction signature digest.

Collecting terms by asset gives

$$K_{\text{bind}} = [\sum_i r_i]R_v + \sum_A [\delta_A]A,$$

where native V is included among the A 's, and

$$\delta_A = \begin{cases} \sum_{i:A_i=V} \Delta_i - b_{\text{nat}}, & A = V, \\ \sum_{i:A_i=A} \Delta_i - b_A, & A \neq V, \end{cases}$$

with $b_A = 0$ when no burn entry exists.

The equality is algebra. The claim that a successful adversary cannot leave some $\delta_A \neq 0$ is computational. If the Action proofs permit extraction of their values, bases, and masks, and the signature argument permits extraction of a scalar k for K_{bind} , an imbalance gives the known relation

$$\sum_A [\delta_A]A + [\sum_i r_i - k]R_v = \mathcal{O}.$$

This contradicts the no-efficient-relation assumption when at least one coefficient is nonzero modulo q .

Binding alone is insufficient for this argument: a binding commitment does not automatically reveal its opening to a reduction. Nor does ordinary fixed-key signature unforgeability by itself establish extraction for an adversarially constructed binding key. The required joint knowledge and composition argument is the same kind of obligation identified in *Ironwood Guide*, §*Balance, digests, and authorisation*, now with multiple asset bases. This volume derives the relation but does not claim a new proof of the complete Halo/signature composition.

4.5 From field balance back to integer balance

The relation argument establishes $\delta_A = 0 \bmod q$. To conclude $\delta_A = 0$ as an integer, prove an absolute bound $|\delta_A| < q$.

If there are at most N Actions, each effective input and output lies in $[0, 2^{64} - 1]$. Therefore

$$\left| \sum_{i:A_i=A} \Delta_i \right| \leq N(2^{64} - 1).$$

For a custom asset, a sufficient bound is

$$N(2^{64} - 1) + (2^{63} - 1) < q.$$

For the native asset use the permitted absolute native balance instead of the custom burn maximum. For example, even the very loose $N \leq 2^{32} - 1$ would make these quantities below 2^{97} , far below the Pallas group order. This is a sufficient arithmetic condition, not a claim that the unspecified carrier actually allows billions of Actions.

A complete transaction format must supply an explicit count or size bound from which this condition follows. Range checks on individual notes do not, by themselves, prevent arbitrarily many field-wrapped terms from summing to zero.

4.6 A two-asset check

Suppose an Action spends 80 units of asset A and creates 50, a second contributes a zero effective input and creates 20 units of A , and the burn set contains $(A, 10)$. The asset equation is

$$(80 - 50) + (0 - 20) - 10 = 0.$$

A separate native Action may pay a fee. Its native balance does not alter this equation.

Replacing the second output with 20 units of another asset B leaves nonzero per-asset coefficients. Even if the ordinary integers appear to add up, the group equation contains different bases. Acceptance would require a useful relation between those bases and the randomness base, unless a proof or range constraint has already failed.

5 Split inputs and the counterfeiting boundary

5.1 Why ordinary zero-value padding is insufficient

An Orchard Action has one input position and one output position. To turn one asset note into two output notes, the transaction needs a second input position. For the native asset, a zero-value dummy input can omit Merkle membership: the native base is a fixed protocol constant and the dummy contributes no money.

If that exception were extended to an arbitrary custom base, the prover could introduce a point with a known relation to V . Consider the hypothetical invalid choice $A_{\text{evil}} = -V$. Create one native output of value w and one custom output of value w , both with dummy zero inputs. Their amount terms are

$$[-w]V + [-w]A_{\text{evil}} = [-w]V + [w]V = \mathcal{O}.$$

The binding-key equation could then hold although the transaction created native value from nothing. All values can be small and in range. The error would be admitting an attacker-chosen base, not overflowing a machine integer.

This is an attack on the *hypothetical weakened relation*, not on the complete split construction. It shows why merely asserting that each Action uses the same base on its two sides is not enough.

5.2 Authenticate the base without consuming value

A split input carries the opening and membership path of an existing custom-asset note, with a private Boolean $s = 1$. The old commitment is opened with its *actual* old value v_o . Only its contribution to the value commitment is zeroed:

$$v'_o = (1 - s)v_o, \quad C = [(1 - s)v_o - v_n]A + [r]R_v.$$

When $s = 0$, the ordinary value equation is recovered. When $s = 1$, changing a large stored v_o cannot create spendable input value because the multiplier is exactly zero.

Let ν be the private native-asset indicator. The relation enforces

$$\nu \in \{0, 1\}, \quad \nu = 1 \iff A = V, \quad s \in \{0, 1\}, \quad s\nu = 0.$$

Thus split inputs are custom-asset inputs, not another native dummy-note encoding.

Write PathOK for the inherited permitted path predicate from *Ironwood Guide*, §*Actions and the proof relation*. It permits noncanonical 255-bit intermediate representatives that reduce to the assigned field values, and the zero-hash alternatives arising from exceptional Sinsemilla addition. It is not merely a canonical path recomputation and root comparison. The old-note membership condition is

$$(v_o = 0 \wedge \nu = 1) \vee \text{PathOK}(\text{cmx}_o, \text{path}, \text{position}, \text{anchor}).$$

All custom inputs require membership, including a zero-valued reference note and a split copy of a previously spent note. Whether the old note’s ordinary nullifier is already spent is irrelevant to its use as a split witness; the split creates a different nullifier and supplies no old value.

The inherited key-consistency and spend-authorisation obligations must also be discharged. A Merkle path and a guessed opening of someone else’s private note are not sufficient. The selected ZIP’s informal freedom to reuse an old note does not remove those inherited proof equations. One can use one’s own available witnesses, obtain the required cooperation, or use the deliberately public reference-note keys. The last option provides a universal usable source without assuming access to arbitrary holders’ keys.

5.3 A fresh nullifier for the non-spending copy

A normal spend uses the note’s fixed nullifier. Reusing that nullifier for every split would either expose repeated use of the same source or cause the spent-nullifier set to reject later splits. The split therefore introduces independent randomness.

Sample a fresh 32-byte seed u and compute

$$\psi_{\text{split}} = \text{ToBase}(\text{PRFexpand}_u(0x0A \parallel \text{enc}_{32}(\rho_o))).$$

Let

$$L = \text{GroupHash}^{\mathbb{P}}(z.\text{cash}:\text{Orchard}, L),$$

and let K be the inherited Orchard nullifier base. The relation uses a witnessed ψ_{nf} constrained by

$$(1 - s)(\psi_{\text{nf}} - \psi_o) = 0.$$

For ordinary inputs this forces the original note randomness; for a split the honest prover uses ψ_{split} . The nullifier equation is

$$\text{nf} = \text{Extract}([(\text{Poseidon}(\text{nk}, \rho_o) + \psi_{\text{nf}}) \bmod p]K + \text{cm}_o + [s]L).$$

The sum inside the brackets is first computed in the Pallas base field, then its canonical integer is used for scalar multiplication. It is not initially an addition modulo the group order q . The Poseidon nullifier PRF and Extract are the inherited functions described in *Ironwood Guide*, §Nullifiers and authenticated existence.

The old commitment still uses ψ_o , not ψ_{split} . Otherwise a prover would change the leaf whose membership it had supposedly demonstrated. For the new note, the inherited Action linkage uses $\rho_n = \text{nf}$.

The L offset is a group-level domain separation. It does not reserve a visibly separate range of nullifier bytes: extraction can map distinct points to the same field value. Independence and collision resistance here are computational properties, not an assertion that normal and split nullifiers occupy disjoint sets.

The seed-to- ψ_{split} expansion is an honest-prover randomness algorithm, not an additional BLAKE2b computation proved inside the Action circuit. The circuit permits a witnessed split ψ_{nf} subject to the stated relation. Consensus still checks every emitted nullifier, including split nullifiers, for duplicates within the transaction and against the branch’s spent-nullifier set.

5.4 Provenance is an inductive invariant

Assume note-commitment binding across the native and custom branches, key-commitment binding for the inherited ownership relation, authenticated tree membership under the permitted representations, and knowledge soundness of the Action proofs. Also assume the Sinsemilla exceptional-case hardness from *Crypto Guide*, §Groups, commitments, and Sinsemilla: efficiently causing the permitted failure output $\perp$, rather than an ordinary commitment or hash, yields an infeasible generator relation.

This assumption is needed both for note/key commitment alternatives and for zero-hash path acceptance. A proof of the weakened relation alone does not remove those alternatives. Outside these explicitly bounded failure events, consider accepted custom-asset notes in their order of creation.

An issue note's base is publicly derived from the issuer and description hash and checked by the issuance validator. Every custom transfer output shares its base with an input note whose membership is proved. That input therefore supplies an earlier committed occurrence of the base. By induction, the base traces to a publicly validated issuance. A split input preserves this argument because its membership condition is unchanged.

The induction concerns *asset origin*, not ownership of all previous notes or preservation of their values. The latter is discharged by the effective-value equation and the binding signature. It also concerns the accepted branch and recognised anchors: a path into an unrelated or rejected tree does not establish provenance.

This separates two obligations that are easy to conflate. Membership prevents introduction of an arbitrary relational base; balance prevents creation of extra units of a legitimate base. Neither condition alone implies the other.

6 The Action relation and transaction validation

6.1 Public statement and private witnesses

The public statement contains the inherited anchor, value commitment, input nullifier, randomised spend-authorisation key, output commitment, and spend/output enable flags, together with the custom-asset enable flag e_A . The asset base and split/native indicators remain private.

The witness contains the old and new note openings, the old-note path and position, the inherited ownership and key-randomisation witnesses, the value-commitment mask, and the additional $A, \nu, s, \psi_{\text{nf}}$. The new circuit must connect these objects, rather than checking disconnected subrelations:

1. Validate each point and scalar in its proper type, and constrain both note values to 64-bit integers.
2. Constrain the inherited diversified-key, incoming-viewing-key, and randomised spend-authorisation-key equations. A split is not a blanket exception to key consistency.
3. Open the old and new note commitments using the one shared asset base and the appropriate native/custom branch, with the inherited exceptional Sinsemilla alternatives.
4. Enforce $\nu = 1$ exactly when $A = V$, require Boolean s, ν , and require $s\nu = 0$.
5. Require PathOK unless it is a native zero-value dummy. Check the conditional normal/split nullifier and the new-note ρ linkage.
6. Enforce the value commitment with effective old value $(1 - s)\nu_O$.
7. Enforce the inherited spend/output enable conditions and require $e_A = 0 \Rightarrow \nu = 1$.

The last rule prevents a prover from hiding custom-asset use in a proof whose public enable flag disables it. Selecting the flag for a transaction or upgrade is a consensus policy obligation, not a choice that can safely be delegated to an unconstrained private witness.

6.2 How the Boolean conditions become field equations

Halo 2 Guide, §*From a relation to a circuit*, explains selectors, constraints, and range checks. Here a Boolean s is enforced by $s(s - 1) = 0$ in the prime field. Similarly enforce $\nu(\nu - 1) = 0$.

Let a_x, a_y be the canonical affine coordinates of the nonidentity A , and v_x, v_y those of V . For the native direction use

$$\nu(a_x - v_x) = 0, \quad \nu(a_y - v_y) = 0.$$

For the non-native direction, witness u_x, u_y and enforce

$$(1 - \nu)((a_x - v_x)u_x - 1)((a_y - v_y)u_y - 1) = 0.$$

When $\nu = 0$, at least one coordinate difference must have an inverse. If both are zero, both parenthesised factors are -1 , so the equation fails. If one is nonzero, its inverse makes the product zero. This enforces inequality of points, not just inequality of their x -coordinates.

In the ordinary nonexceptional path calculation, let r_{path} be the computed root. The membership gate is

$$(v_o + 1 - \nu)(r_{\text{path}} - \text{anchor}) = 0.$$

Because v_o is range-constrained and $2^{64} < p$, the first factor is zero exactly when $v_o = 0, \nu = 1$. Without the range check it could also vanish through an unintended field value. This gate explains the native-dummy bypass; the complete inherited PathOK predicate retains the representation and zero-hash allowances stated above. It is not strengthened to canonical root equality by this illustrative equation.

The inherited enable conditions are

$$v_o(1 - e_{\text{spend}}) = 0, \quad v_n(1 - e_{\text{output}}) = 0,$$

and the new condition is

$$(1 - e_A)(1 - \nu) = 0.$$

Notice that the spend-enable equation uses the *old note value*, not its zeroed split contribution. Copying a nonzero old note still requires spends enabled under the inherited condition. A zero-valued public reference avoids that particular nonzero-value requirement.

These equations explain the new gates. They do not replace the elliptic-curve, Sinsemilla, Merkle, and key-consistency gadgets they connect, or prove that a particular compiled circuit has wired them all correctly.

6.3 Complete validation order at the defined interface

A validator must combine the following checks, using the generic atomic state-transition contract in Consensus and the inherited payment checks in *Ironwood Guide*, §Pool rules and complete validation.

1. Select the rules for the containing branch and activation context. Parse canonical transaction and field encodings, reject inconsistent counts, and enforce resource bounds. Under the selected ZIP 227 rule, the Action Group count is zero or one and its group expiry height is zero.
2. Check the bundle flags, recognised anchor, and every nullifier's uniqueness within the transaction and against the branch state. A split nullifier is not exempt.
3. Reconstruct the exact public proof instances from the transaction fields and verify the proof under the matching circuit/verifying key.
4. Compute the transaction signature digest and verify every spend-authorisation signature under its randomised key. Recompute the multibase binding key and verify its binding signature.
5. Check the distinct public burn entries and run the burn-first issuance-state transition. If issuance is present, verify its issuer signature, reconstruct every issue note, and check the reference, supply, and finalisation rules.

6. Check the inherited native transaction balance and fee conditions, as well as all remaining contextual transaction rules.
7. Commit all state changes together: spent nullifiers, transfer and issuance commitment leaves in the specified order, and the new issuance map. Otherwise change none of them.

The expensive cryptographic checks can be batched or reordered operationally when that preserves acceptance semantics. The state effects cannot be partially applied merely because an earlier check succeeded.

6.4 What is still needed for a deployed endpoint

Three unresolved interfaces in the selected pin prevent the above from being a complete network implementation.

1. *Carrier and digests.* A non-withdrawn transaction encoding must frame all transfer, issuance, and burn fields, fix count/size limits, assign branch/version identifiers, and specify which exact bytes every signature and authorising commitment covers.
2. *Note-format instantiation.* The ZSA plaintext lead byte, note-randomness derivation, encrypted plaintext lengths, and incoming/outgoing decryption behaviour must agree. In particular, validators must deterministically reconstruct issue-note commitments from the public fields.
3. *Circuit and activation.* The complete relation must be implemented and its verifying key selected by explicit consensus rules, including the meaning of the enable flag and exceptional invalid asset-generation outcomes.

The new group equations do not fill these gaps automatically. Nor do they supply a new proof of the complete multibase knowledge-soundness and authorisation composition.

Within the specified interface, the critical path is now explicit: issuer identity determines a hashed asset base; public issuance places that base in authenticated notes; every custom transfer preserves such provenance; constrained value commitments and authorisation enforce conservation; public burns and issuance update a branch-local supply map. Every step has a distinct cryptographic or state-machine obligation, and no step can be replaced by trusting a displayed asset name.